

Lab02

Mateus Bortolucci

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(readr)  
library(ggplot2)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)? $T(X)$: A contagem absoluta de voos onde $ARRIVAL_DELAY > 10$. n : O número total de voos analisados. O percentual estimado será simplesmente o estimador de máxima verossimilhança: $\hat{p} = \frac{T(X)}{n}$.
2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
 - e. A função deve receber apenas dois argumentos:
 - f. input: o conjunto de dados (referente ao lote em questão);

- ii. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  
  resultado <- input %>%  
    # a.  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
  
    # b.  
    filter(!is.na(AIRLINE) & !is.na(MONTH) & !is.na(DAY) & !  
is.na(ARRIVAL_DELAY)) %>%  
  
    # c.  
    group_by(MONTH, DAY, AIRLINE) %>%  
  
    # d.  
    summarise(  
      tamanho_amostra_n = n(),  
      voos_atrasados_Tx = sum(ARRIVAL_DELAY > 10),  
      .groups = "drop"  
    )  
  
  return(resultado)  
}
```

3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip.
- Configure o tamanho do lote (chunk) para 100 mil registros;
 - Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;
 - Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
colunas_interesse <- cols_only(  
  MONTH = col_integer(),  
  DAY = col_integer(),  
  AIRLINE = col_character(),  
  ARRIVAL_DELAY = col_double()  
)  
estatisticas_consolidadas <- read_csv_chunked(  
  file = "flights.csv",  
  callback = DataFrameCallback$new(getStats),  
  chunk_size = 100000,  
  col_types = colunas_interesse  
)
```

```
head(estatisticas_consolidadas)
```

```
# A tibble: 6 × 5
  MONTH   DAY AIRLINE tamanho_amostra_n voos_atrasados_Tx
  <int> <int> <chr>          <int>          <int>
1     1     1    AA             1379             514
2     1     1    DL             1558             132
3     1     1    UA             1269             367
4     1     1    US             1028             201
5     1     2    AA             1508             649
6     1     2    DL             2261             406
```

4. Crie uma função chamada `computeStats` que:
 - a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
 - c. Cia: sigla da companhia aérea;
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats <- function(dados_consolidados) {
  resultado_final <- dados_consolidados %>%
    group_by(AIRLINE, MONTH, DAY) %>%
    summarise(
      n_total = sum(tamanho_amostra_n),
      Tx_total = sum(voos_atrasados_Tx),
      .groups = "drop"
    ) %>%
    mutate(
      Cia = AIRLINE,
      Data = as.Date(paste("2015", MONTH, DAY, sep = "-")),
      Perc = Tx_total / n_total
    ) %>%
    select(Cia, Data, Perc)

  return(resultado_final)
}
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes `ggcal` e `ggplot2`.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
- d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
- ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
- iii. Retornar para o usuário a base do calendário criada acima.
- iii. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```
# 1. Carregar os pacotes necessários
library(ggplot2)
library(dplyr)

# 2. Declarar a função ggcal (substituindo o pacote que saiu do ar)
ggcal <- function(x, y) {
  df <- data.frame(Data = as.Date(x), Valor = y)

  # Cria rótulos ordenados para os dias da semana e meses
  df$DiaSemana <- factor(format(df$Data, "%w"),
                        levels = 0:6,
                        labels = c("Dom", "Seg", "Ter", "Qua", "Qui", "Sex",
                                   "Sáb"))

  df$Mes <- factor(format(df$Data, "%m"),
                  levels = sprintf("%02d", 1:12),
                  labels = c("Jan", "Fev", "Mar", "Abr", "Mai", "Jun",
                             "Jul", "Ago", "Set", "Out", "Nov", "Dez"))

  # Extrai a semana do ano para alinhar o eixo Y do calendário
  df$SemanaAno <- as.numeric(format(df$Data, "%U"))

  # Monta o calendário base
  grafico <- ggplot(df, aes(x = DiaSemana, y = -SemanaAno, fill = Valor)) +
    geom_tile(color = "white") +
    facet_wrap(~ Mes, scales = "free", ncol = 4) +
    theme_minimal() +
    theme(
      axis.text.y = element_blank(),
      axis.title = element_blank(),
      panel.grid = element_blank(),
      axis.ticks = element_blank(),
      strip.text = element_text(face = "bold", size = 10)
    )
}
```

```

    )

    return(grafico)
}

# 3. Definir a paleta de cores gradiente no objeto 'pal'
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027", name = "Atraso")

# 4. Criar a função baseCalendario
baseCalendario <- function(stats, cia) {
  # a. Criar um subconjunto apenas com as informações da Cia. Aérea dada
  sub_stats <- stats %>% filter(Cia == cia)

  # b. Montar a base do calendário utilizando ggcal(x, y)
  calendario_base <- ggcal(x = sub_stats$Data, y = sub_stats$Perc)

  # c. Retornar a base do calendário
  return(calendario_base)
}

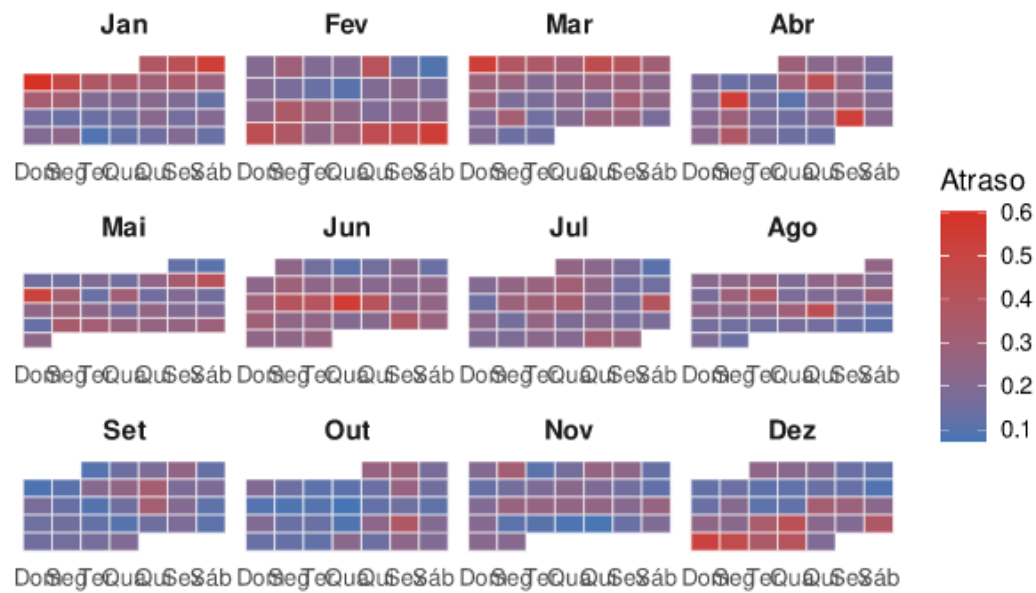
dados_finais <- computeStats(estatisticas_consolidadas)

cAA <- baseCalendario(stats = dados_finais, cia = "AA")
cDL <- baseCalendario(stats = dados_finais, cia = "DL")
cUA <- baseCalendario(stats = dados_finais, cia = "UA")
cUS <- baseCalendario(stats = dados_finais, cia = "US")

cAA + pal + ggtitle("Percentual de Atrasos (ARRIVAL_DELAY > 10) - American
Airlines (AA)")

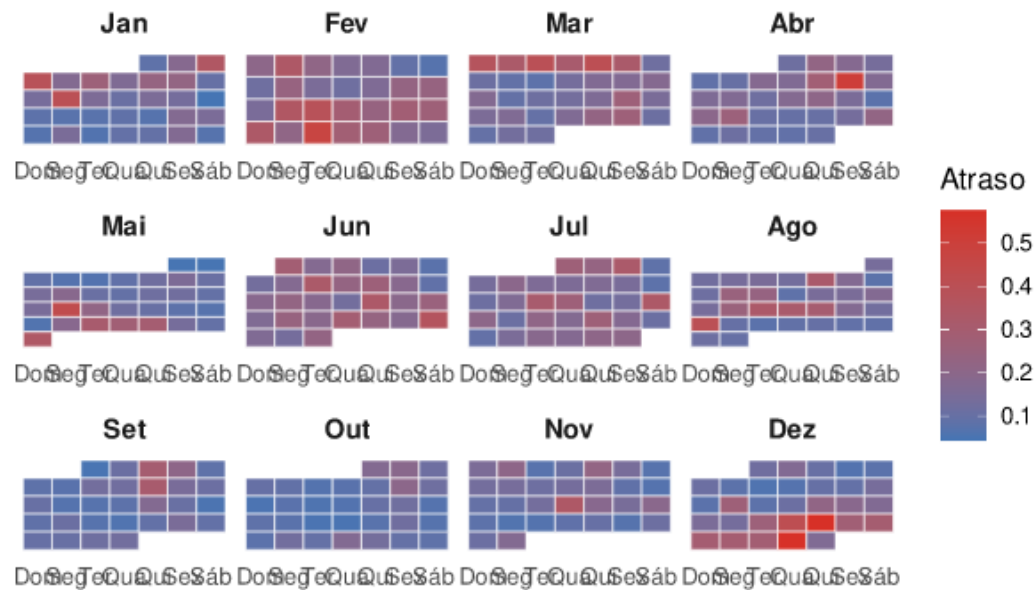
```

Percentual de Atrasos (ARRIVAL_DELAY > 10) - American Airlines



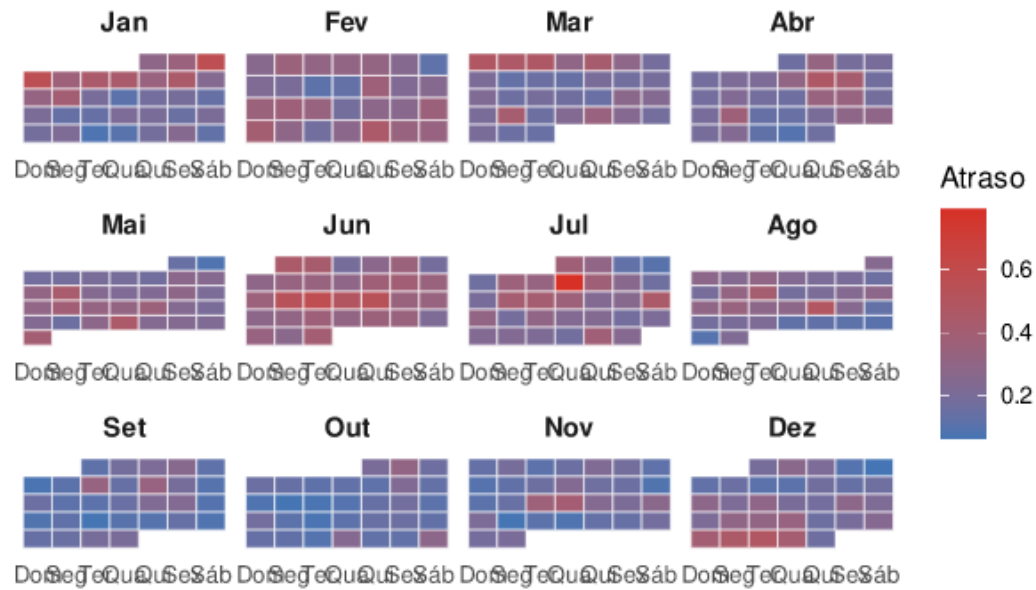
```
cDL + pal + ggtitle("Percentual de Atrasos (ARRIVAL_DELAY > 10) - Delta Air Lines (DL)")
```

Percentual de Atrasos (ARRIVAL_DELAY > 10) - Delta Air Lines (DL)



```
cUA + pal + ggtitle("Percentual de Atrasos (ARRIVAL_DELAY > 10) - United Airlines (UA)")
```

Percentual de Atrasos (ARRIVAL_DELAY > 10) - United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos (ARRIVAL_DELAY > 10) - US Airways (US)")
```

Percentual de Atrasos (ARRIVAL_DELAY > 10) - US Airways (US)

